AI OPERATIONS (AIOps)

Module 4 — Assignment 5 (Student)

SwiftParcel: serve a delivery-time (ETA) model four ways · 81 points · CPU only

1. Scenario

SwiftParcel, a regional courier, predicts the **delivery time in minutes** (a *regression*, unlike the churn classifier from the lecture) from seven features of a delivery. The data science team has trained two models — **v1**, a Ridge regression, and **v2**, a gradient-boosting regressor — and now needs you to turn them into a production-grade service. You will build the service, manage and release its versions, add a smarter canary, and gate its release with a smoke test. The lecture notebooks use a different problem on purpose: reusing the ideas is the point, copying the cells will not be enough.

Ground rules

Individual work. You may reuse *ideas* and small fragments from the lecture notebooks, but cite them in your report. Do not share code.

CPU only; everything must run on a laptop with 2+ cores. Python 3.10+.

Hardware-dependent numbers (latency, throughput) are graded on **method and reasoning**, never against a fixed value. State your machine (CPU cores, RAM).

Be honest about what you did not run. A clearly labelled 'not tested' is never penalised; a claim that was not verified is.

Due one week after the lecture, as a single zip (see §7).

2. What you are given

File	Purpose
eta_data.py	`python eta_data.py` writes `data/eta_train.csv`, `data/eta_fresh.csv` and the two model bundles `data/eta_v1.joblib` (Ridge) and `data/eta_v2.joblib` (GBR). Exposes `make_data()` and `FEATURES`.
load_test.py	The load generator from the lecture. Use `--data data/eta_sample.csv --url … --concurrency … --batch ….`
requirements.txt	Suggested versions.
README.md	Setup commands.

Each model bundle is a joblib dict `{model, feature_columns, version}`. The seven features are `distance_km, n_stops, hour, weekday, traffic_index, rain, weight_kg`.

2.1 Interface contract (so that we can run automatic checks)

Part	File	Names we will import / run
A	eta_app.py	A FastAPI object named `app`; model path from environment variable `ETA_MODEL` (default `data/eta_v1.joblib`); module-level variable `model`
A	test_eta_app.py	Your own pytest file (we also run ours against your app)
B	eta_mlflow.py	Functions `register()`, `decide(prod, cand, **thresholds)`, `promotion_gate(**thresholds)`; tracking URI from `MLFLOW_TRACKING_URI`
B	test_gate.py	Your pytest file for `decide()`
C	eta_ray.py	Function `build(weight=0.1, extra_work_ms=0.0)` returning a Serve application; route POST `/v1/models/eta:predict`
C	verify_ray.py	Your verification script
D	smoke_eta.py	Command line `--url`, `--expect-version`, `--max-p95-ms` (see Part D)

3. Part A — REST service (25 points)

A1. Build `eta_app.py` (12 points)

Implement a FastAPI service with these requirements:

1. `POST /v1/models/eta:predict` accepts `{"instances": [ {7 features}, … ]}` and returns `{"predictions": [minutes, …], "model_version": …, "request_id": …}`. Predictions are rounded to one decimal and returned in the same order as the inputs.
2. Ranges: `distance_km` 0.1–500; `n_stops` 1–60; `hour` 0–23; `weekday` 0–6; `traffic_index` 0–1; `rain` 0 or 1; `weight_kg` > 0 and ≤ 1000. Batch size 1–500.
3. A **cross-field rule**: a delivery with `distance_km > 150` **and** `n_stops > 20` is infeasible and must be rejected with HTTP 422 and a message containing the word `infeasible`.
4. Every response (success **and** error) carries an `X-Request-ID` header: echo the client's header if present, otherwise generate a UUID. The same id appears as `request_id` in the JSON body.
5. Validation errors return 422 with the body `{"error": "validation_failed", "details": [{"field": …, "msg": …}], "request_id": …}`.
6. `GET /metrics` returns Prometheus text: a counter `eta_requests_total{path=…,status=…}` and a latency histogram `eta_request_latency_seconds` (several `_bucket{le=…}` lines including `+Inf`, plus `_sum` and `_count`). Requests to `/metrics` itself are not counted.
7. `GET /healthz` returns `{"status":"ok","model_version":…}`. If inference raises, return a **generic** 500 (no internals in the body).

A2. Tests (5 points)

Write `test_eta_app.py` with at least **10** pytest cases using FastAPI's `TestClient`. They must cover at least: a valid request; batch order preserved; three different kinds of invalid input; the cross-field rule; both batch-size limits; request-id echo and generation (including in an error); the metrics output; and a generic 500 (use `monkeypatch` to make the model raise).

A3. Load test and report (8 points)

1. Serve v1. Run `load_test.py` at concurrency 1, 2, 4, 8, 16 (4 s each) and put the table in your report.
2. State the **saturation point** and your service's **capacity under the SLO p95 ≤ 80 ms**, and show how you decided.
3. Measure the cost per row of batch size 1 versus batch size 100 at concurrency 1. Report ms/row for both and explain the difference.
4. Propose **one** change that should roughly double capacity on your machine, apply it, measure, and report the result honestly — including if it did not double.
5. In two sentences: why do we report p95/p99 rather than the mean?

4. Part B — MLflow: registry, gate and shadow (25 points)

B1. Register (8 points)

In `eta_mlflow.py`, write a custom `pyfunc` that wraps a bundle and returns a DataFrame with columns `eta_minutes` (one decimal) and `model_version`. Log both models with a **signature** and an **input example**, pinned `pip_requirements`, and register them as `eta-model` with aliases `prod` → v1 and `candidate` → v2. Include the registry table (version, alias) in your report. Use a SQLite tracking store.

B2. A multi-criteria promotion gate (9 points)

Implement a **pure** function `decide(prod, cand, min_rel_gain=0.03, max_p95_ms=10.0, max_slice_regression=0.05)` that receives two metric dictionaries with keys `mae`, `mae_rush`, `mae_rain`, `p95_ms` and returns `{"criteria": {…}, "promote": bool}`. Promote only if **all** hold:

- overall MAE improves by at least `min_rel_gain` (relative to prod);
- the candidate's single-row inference p95 is at most `max_p95_ms`;
- **no slice regresses**: MAE on rush-hour rows (hours 7–9 and 16–19) and on `rain == 1` rows must not exceed prod's by more than `max_slice_regression`.

Then implement `promotion_gate(**thresholds)` that loads both aliases, evaluates them on **fresh data** (`eta_data.make_data(2000, seed=99)`), measures p95 over 200 single-row predictions, and calls `decide`. Write `test_gate.py` with at least **6** tests of `decide()` using synthetic numbers (include boundary cases just inside and just outside a threshold). Report the gate's output with default thresholds and with `min_rel_gain=0.60`.

B3. Shadow comparison (8 points)

Serve `prod` on port 5101 and `candidate` on port 5102 with `mlflow models serve`. Send the **same** 200 requests (from `data/eta_fresh.csv`, one row each) to both. Report (i) the mean absolute difference between their predictions in minutes, (ii) the percentage of requests whose predictions differ by more than 10 minutes, and (iii) each server's p95 latency. In about five sentences state whether you would promote, and answer: *why does moving an alias not change what an already running server returns, and what are two ways a running service could pick up the new version without dropping requests?*

5. Part C — Ray Serve: a sticky canary (25 points)

C1. Build `eta_ray.py` (12 points)

1. A deployment `EtaModel` that loads a bundle once per replica and predicts a batch of rows. `prod` runs **2 replicas**; the `candidate` uses an **autoscaling_config** (min 1, max 3).
2. A router (FastAPI ingress) that receives `{"customer_id": "…", "instances": […]}` and returns the model's output plus `"arm": "v1" | "v2"`.
3. **Sticky canary:** a given `customer_id` must always be routed to the same arm, and about `weight` of customers (not requests!) must go to the candidate. Use a **stable hash** of the id.
4. Header override: `X-Beta: 1` forces the candidate for that request.
5. Replicas reserve fractional CPUs so the whole application fits on 2 CPUs. Expose `build(weight=0.1, extra_work_ms=0.0)`.

C2. Verification script (8 points)

Write `verify_ray.py` that deploys your app on a 2-CPU Ray and demonstrates, with assertions and printed evidence, using 1000 distinct customer ids and `weight = 0.2`: (a) the share routed to the candidate is within ±4 percentage points of 0.2; (b) 200 customers each get the same arm on a second call; (c) the same holds after **redeploying** the identical app; (d) after raising the weight from 0.2 to 0.5, **no customer who was on the candidate moves back** to prod; (e) the header override works for 50 requests.

C3. Short answers (5 points)

1. Why is Python's built-in `hash()` the wrong tool for the bucketing function here?
2. Give two reasons stickiness matters for an ETA product *and* for analysing the experiment.
3. Justify your autoscaling parameters (`target_ongoing_requests`, delays, max replicas) in 3–4 sentences.

6. Part D — Release gate (6 points)

Before a new version takes real traffic, an automated check should decide whether it is allowed to. You can run it against any of your own servers (Part A, B or C).

D1. Smoke test and abort criteria

1. `smoke_eta.py --url … --expect-version … [--max-p95-ms 80]`: exit 0 on success and 1 on failure, with at least **five** checks, of which at least two are *model-sanity* checks (e.g. a longer distance must not give a shorter ETA; heavier traffic must not shorten it) and one checks that invalid input returns 422.
2. Four **abort criteria** you would monitor during a 10 % canary of v2 (be specific: metric, threshold, window).

7. Submission

```
Module4_<rollno>.zip
  REPORT.md  (or .pdf)         <- all tables and written answers, machine description, list
of anything NOT tested
  partA/ eta_app.py  test_eta_app.py  loadtest_results.json
  partB/ eta_mlflow.py  test_gate.py  gate_output.txt  shadow_output.txt
  partC/ eta_ray.py  verify_ray.py  verify_output.txt
  partD/ smoke_eta.py  (abort criteria go in REPORT.md)
```

Your report should be at most four pages of prose plus tables. Do **not** include model files or databases; we regenerate them with `eta_data.py`.

8. Evaluation rubric

Each criterion is scored by level. Partial credit applies throughout; automated checks and manual review are both used.

Part	Criterion	Points	What earns full marks
A1	Correct, complete service	12	All seven requirements pass our automated tests; code is readable and loads the model once
A2	Test suite	5	≥ 10 meaningful tests covering every listed area; they fail when the behaviour is broken
A3	Load test and reasoning	8	Correct table; saturation reasoned with Little's Law; capacity under SLO justified; honest measured result for the change; percentile rationale
B1	Registration	8	pyfunc output as specified; signature + input example; two versions; aliases; table shown
B2	Gate	9	Pure `decide()` correct incl. slices; `promotion_gate` uses fresh data; ≥ 6 boundary-aware tests; both outputs reported
B3	Shadow + reasoning	8	Three metrics measured on identical requests; sound decision; correct explanation of alias resolution and two valid ways a running service can pick up the new version
C1	Sticky canary app	12	Passes automated checks: share, stickiness, header, autoscaling config, fractional CPU
C2	Verification script	8	Five checks with assertions and printed evidence
C3	Short answers	5	Correct on hash() salting, stickiness, parameter justification
D1	Release gate	6	Smoke test with ≥ 5 checks incl. model-sanity and 422; four specific abort criteria (metric, threshold, window)

Cross-cutting deductions (applied once, up to −10 in total)

- Claims of results that were not run, or numbers that cannot come from your code: up to −5.
- Missing machine description or missing 'not tested' list: −2.
- Code that cannot be run from a clean checkout following your own instructions: up to −5.
- Copied code or text without citation: handled under the academic-integrity policy.